香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 22

File System 3:
buffering & reliability

Prof. Yunming XIAO

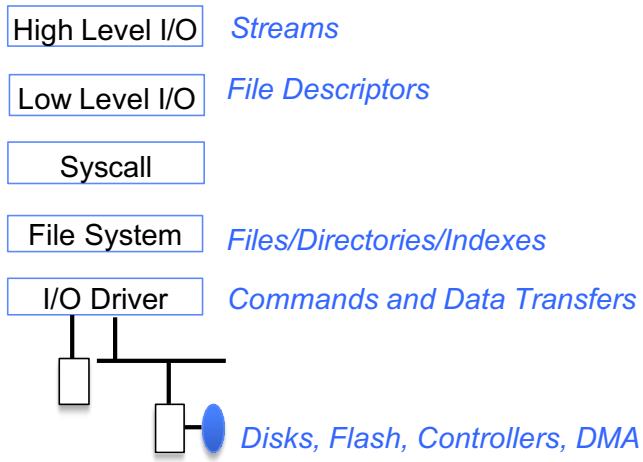
School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Recall

Recall: file systems

- Take limited hardware interface (array of blocks) and provide a more convenient/useful interface with:
  - Naming: Find file by name, not block numbers
  - Organization: Organize file names within directories
  - Placement: Map files to blocks
  - Protection: Enforce access restrictions
  - Reliability: Keep files intact despite crashes, failures, etc.

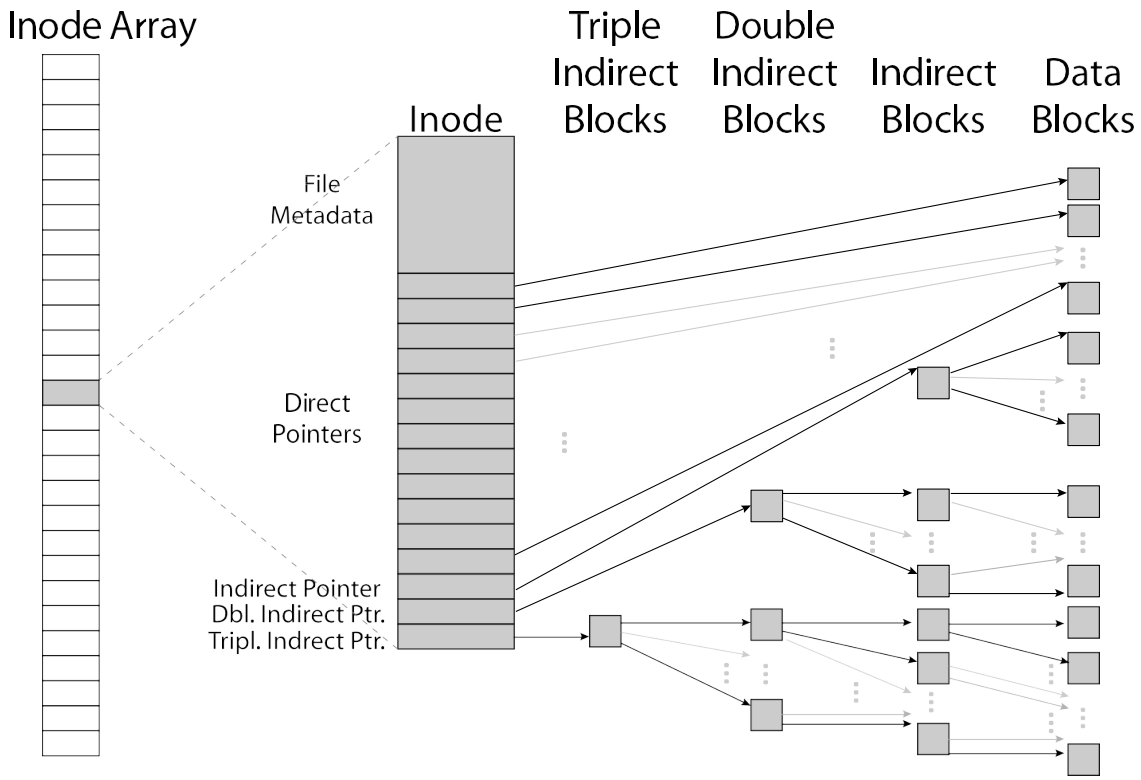


Recall: Unix file system

- Superblock object: information about file system
- Free bitmaps: what is allocated/not allocated
- Inode object: represents a specific file (“index node”)
- Dentry object: directory entry, single component of a path
- Blocks: How files are stored on disk

Same concepts exist in many current file systems, such as Linux ext4 and Mac APFS

Recall: inode data structure



Improving the Unix FS: Berkeley Fast File System

A Fast File System for UNIX*

Marshall Kirk McKusick, William N. Joy†,
Samuel J. Leffler‡, Robert S. Fabry

Computer Systems Research Group
Computer Science Division
Department of Electrical Engineering and Computer Science
University of California, Berkeley
Berkeley, CA 94720

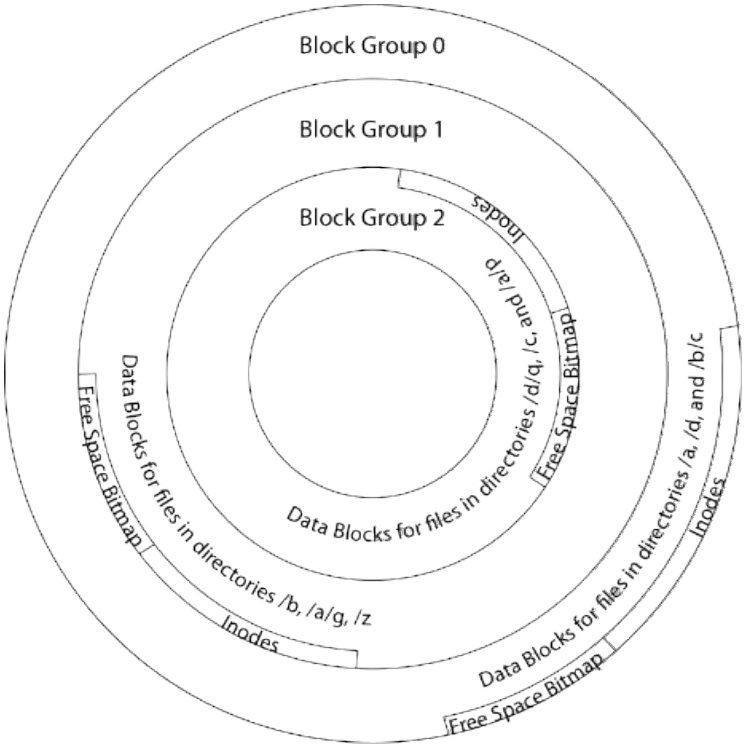
ABSTRACT

A reimplementation of the UNIX file system is described. The reimplementation provides substantially higher throughput rates by using more flexible allocation policies that allow better locality of reference and can be adapted to a wide range of peripheral and processor characteristics. The new file system clusters data that is sequentially accessed and provides two block sizes to allow fast access to large files while not wasting large amounts of space for small files. File access rates of up to ten times faster than the traditional UNIX file system are experienced. Long needed enhancements to the pro-

Added “disk-aware” optimizations

FFS locality: block groups

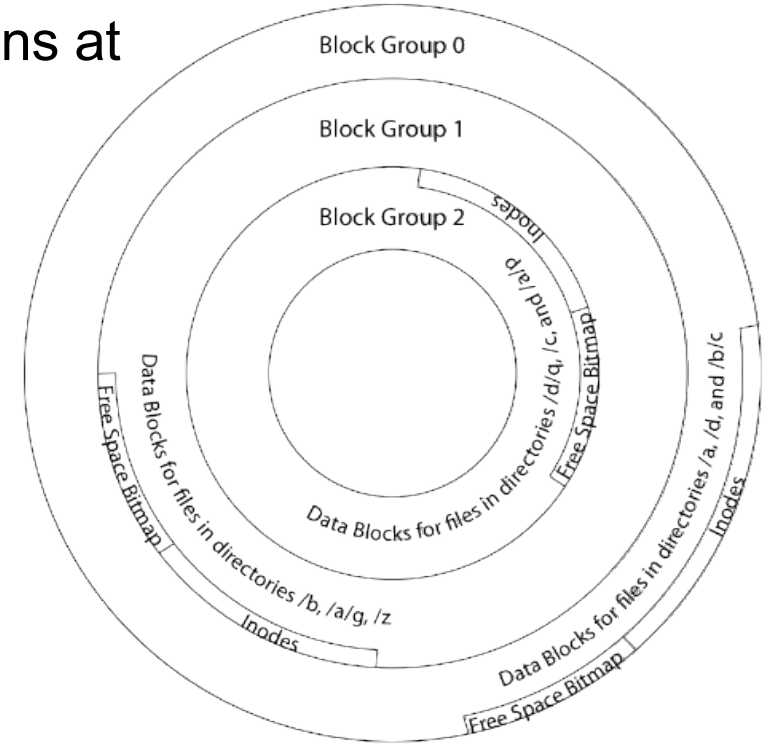
- Distribute header information (inodes) closer to the data blocks, in same “cylinder group”
- File system volume divided into set of “block groups”
- Data blocks, metadata, and free space interleaved within block group
- Put a directory and its files in same block group



FFS locality: block groups

- First-Free allocation of new file blocks
  - To expand file, first try successive blocks in bitmap, then choose a new range of blocks
  - Few little holes at start, big sequential runs at end of group
  - Avoids fragmentation
  - Sequential layout for big files

- Important: keep 10% or more free!
  - Reserve space in the Block Group



Attack of the rotational delay

- Missing blocks due to rotational delay
- Issue: Read one block, do processing, and read next block. In meantime, disk has continued turning: missed next block! Need 1 revolution/block!
- Solution 1: Skip sector positioning ("interleaving")
  - Place the blocks from one file on every n^{th} block of a track: give time for processing to overlap rotation
  - Can be done by OS (supported in Unix FFS) or possibly in disk controller
- Solution 2: Read-ahead: read next block right after first, even if application hasn't asked for it; again, can be done by OS or disk controller (in its device RAM)

UNIX 4.2 BSD FFS

- Pros
  - Efficient storage for both small and large files
  - Locality for both small and large files
  - Locality for metadata and data
  - No defragmentation necessary!

- Cons
  - Inefficient for tiny files (a 1 byte file requires both an inode and a data block)
  - Inefficient encoding when file is mostly contiguous on disk
  - Need to reserve at least 10% of free space to prevent fragmentation

What about other file systems?

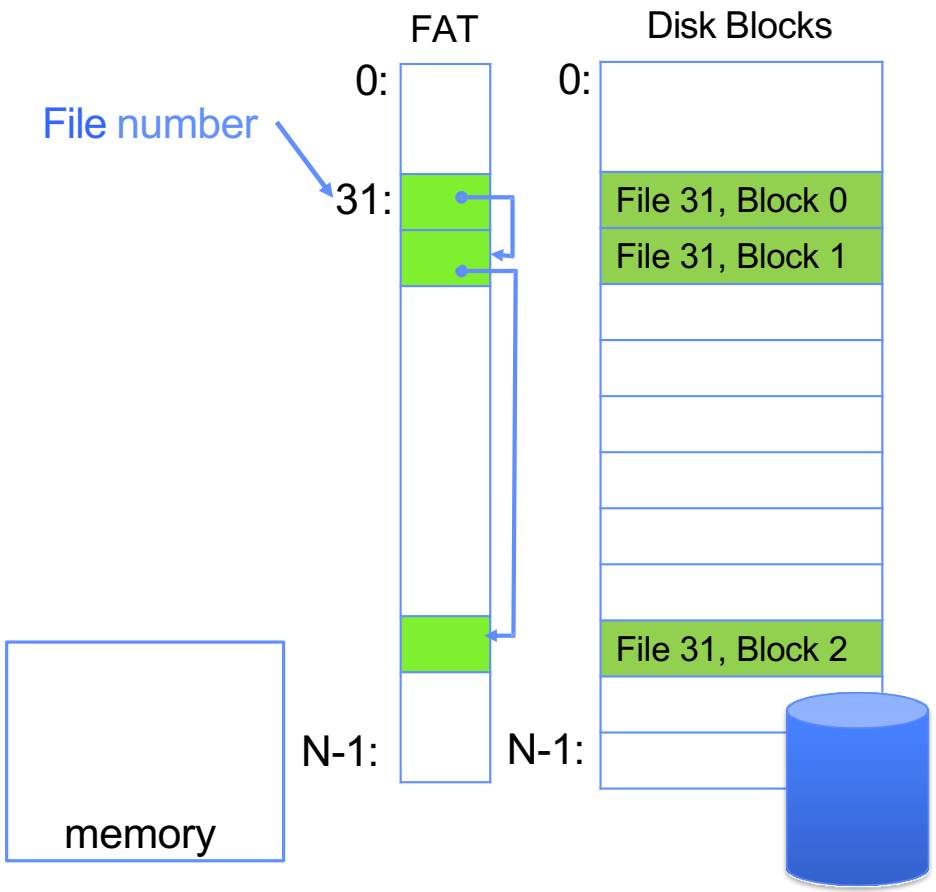
FAT:
File Allocation Table

(MS-DOS,1977)

Windows NTFS
(1993 – now)

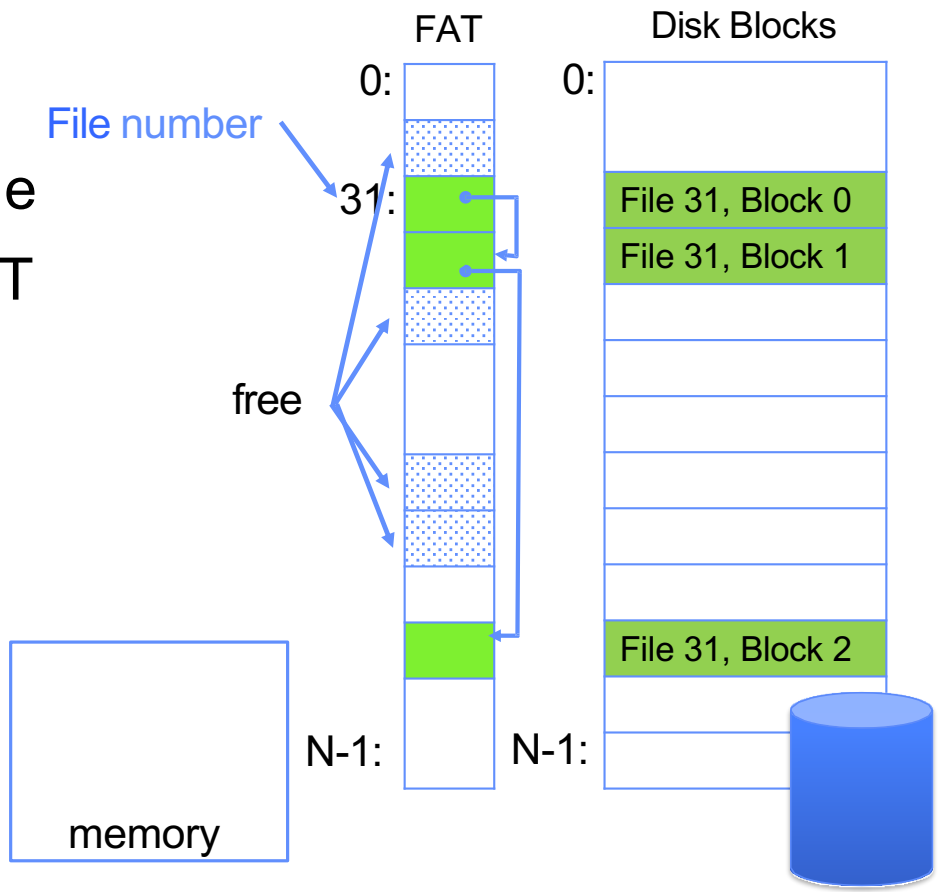
FAT (File Allocation Table)

- Disk holds a File Allocation Table (FAT) plus an array of data blocks, both same length
  - One FAT entry per data block
  - FAT entries for same file form a linked list
- Example: file_read 31, < 2, x >
  - Index into FAT with file number (31)
  - Follow linked list to block 2
  - Read the block from disk into memory

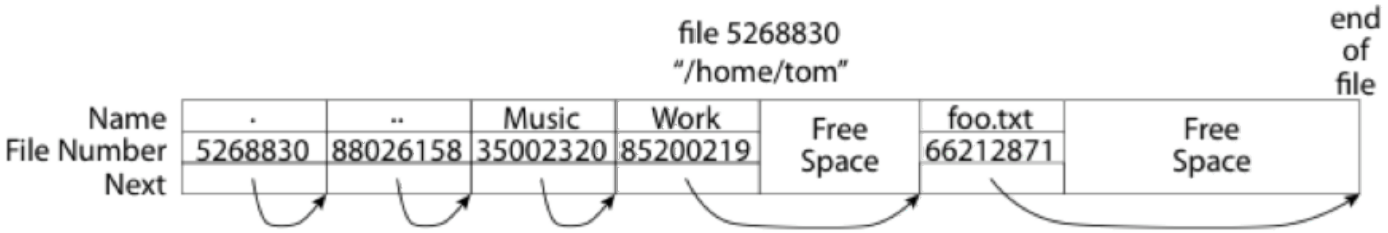


FAT (File Allocation Table)

- File number is ID of the file's first block
- Follow list to get other blocks in file
- Unused blocks marked free in FAT
  - Could require scan to find one
  - Or, could use a free list



FAT: directories

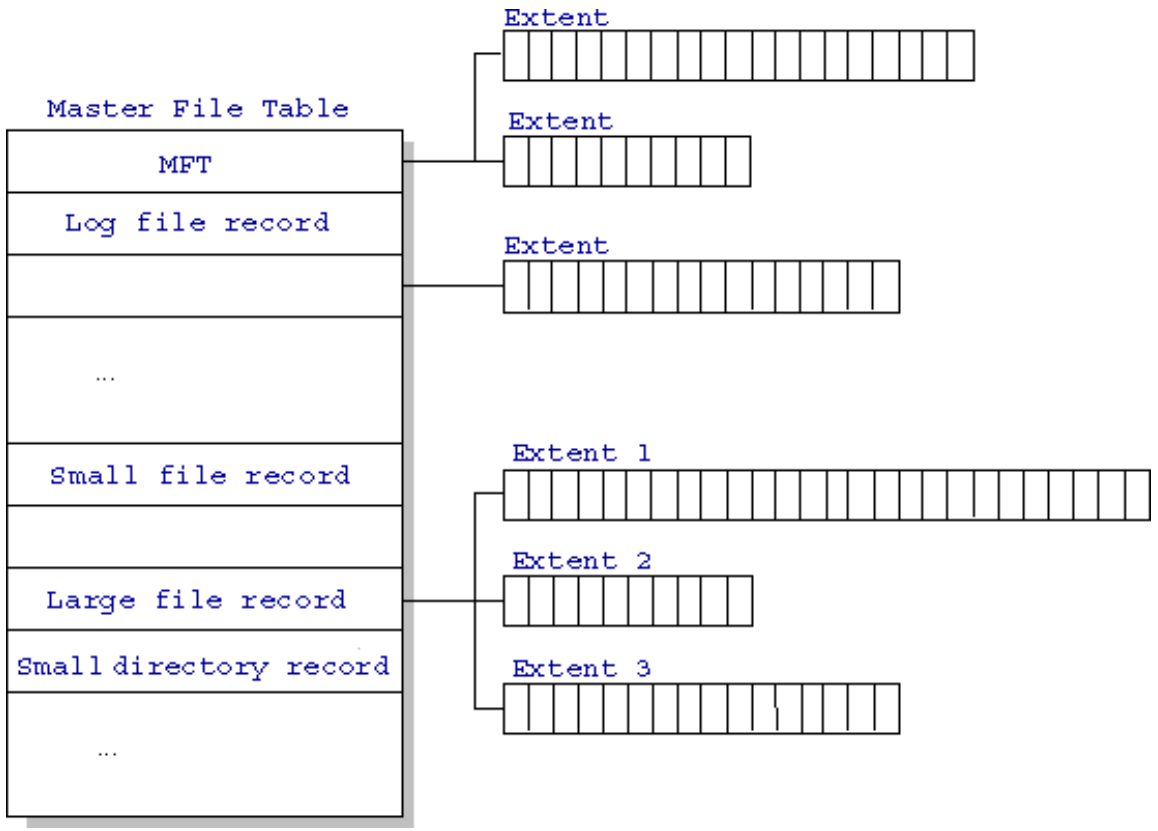


- A directory is a file containing <file_name: file_number> mappings
- In FAT: file attributes are kept in directory (!!!)
  - Not directly associated with the file itself
- Each directory a linked list of entries
  - Requires linear search of directory to find particular entry
- Where do you find root directory ("/")?
  - At well-defined place on disk
  - For FAT, this is at block 2 (there are no blocks 0 or 1)

New Technology File System (NTFS)

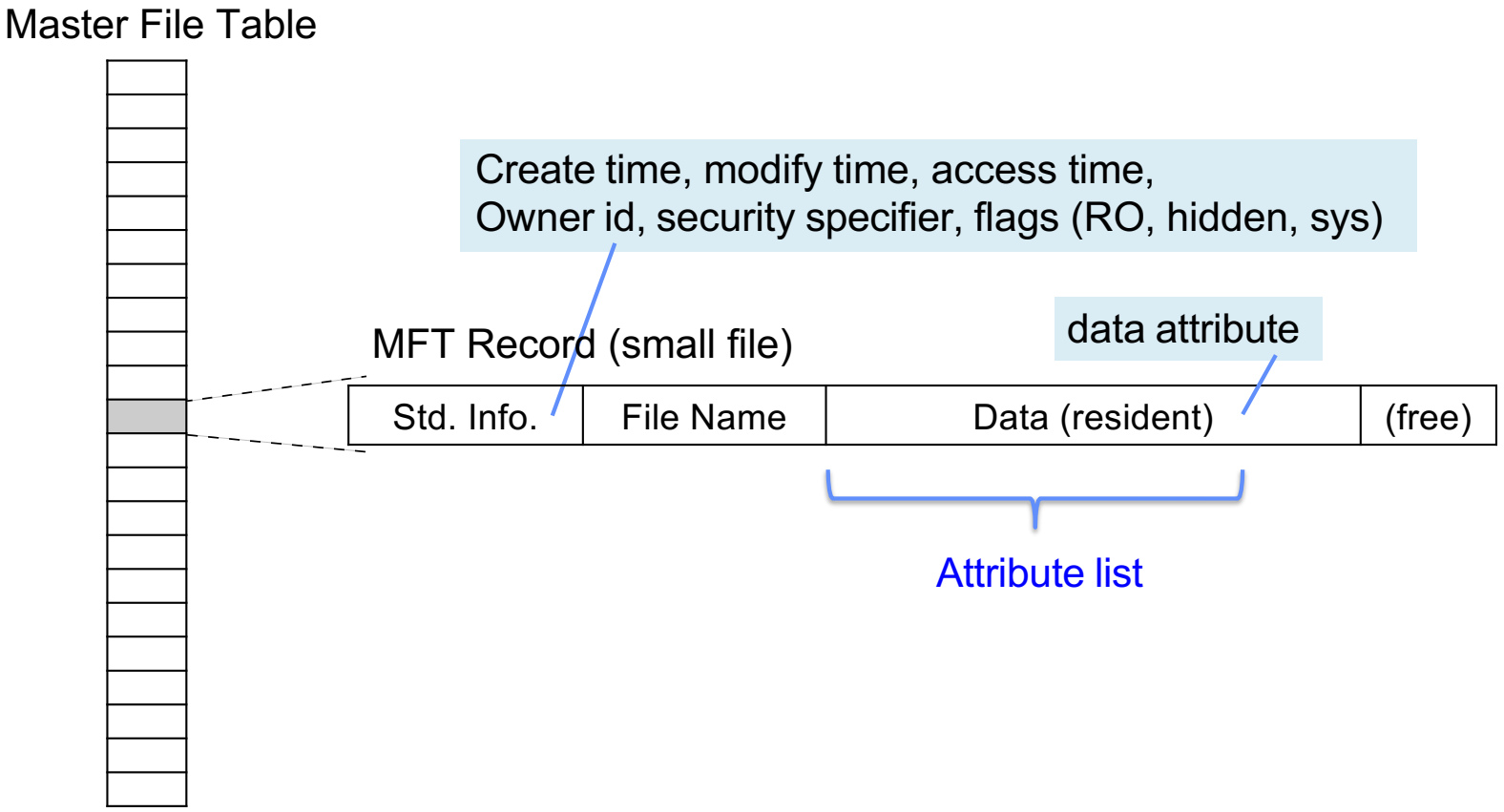
- Default on modern Windows systems
- Variable length extents
- Master File Table
  - Everything (almost) is a sequence of <attribute:value>
- Each entry in MFT contains metadata and:
  - File's data directly (for small files)
  - A list of extents (start block, size) for file's data
  - For big files: pointers to other MFT entries with more extent lists

NTFS

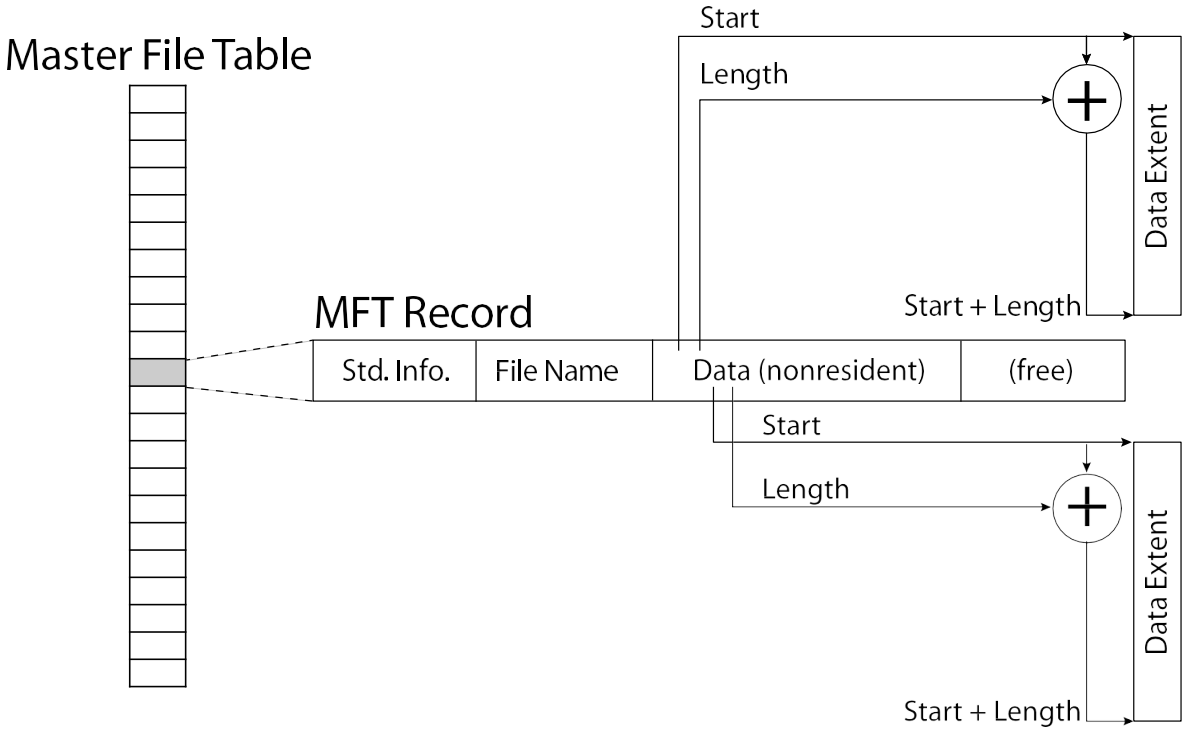


http://ntfs.com/ntfs-mft.htm

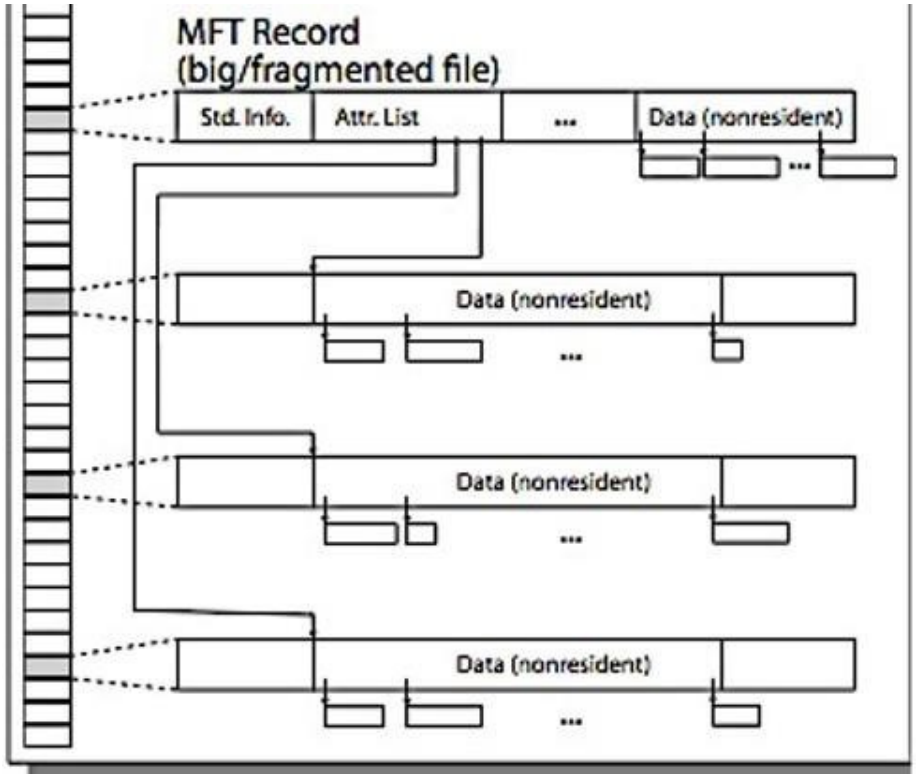
NTFS small file: data stored with metadata



NTFS medium file: extents for file data



NTFS large file: pointers to other MFT



Buffer cache

Buffer caches: speeding up file systems

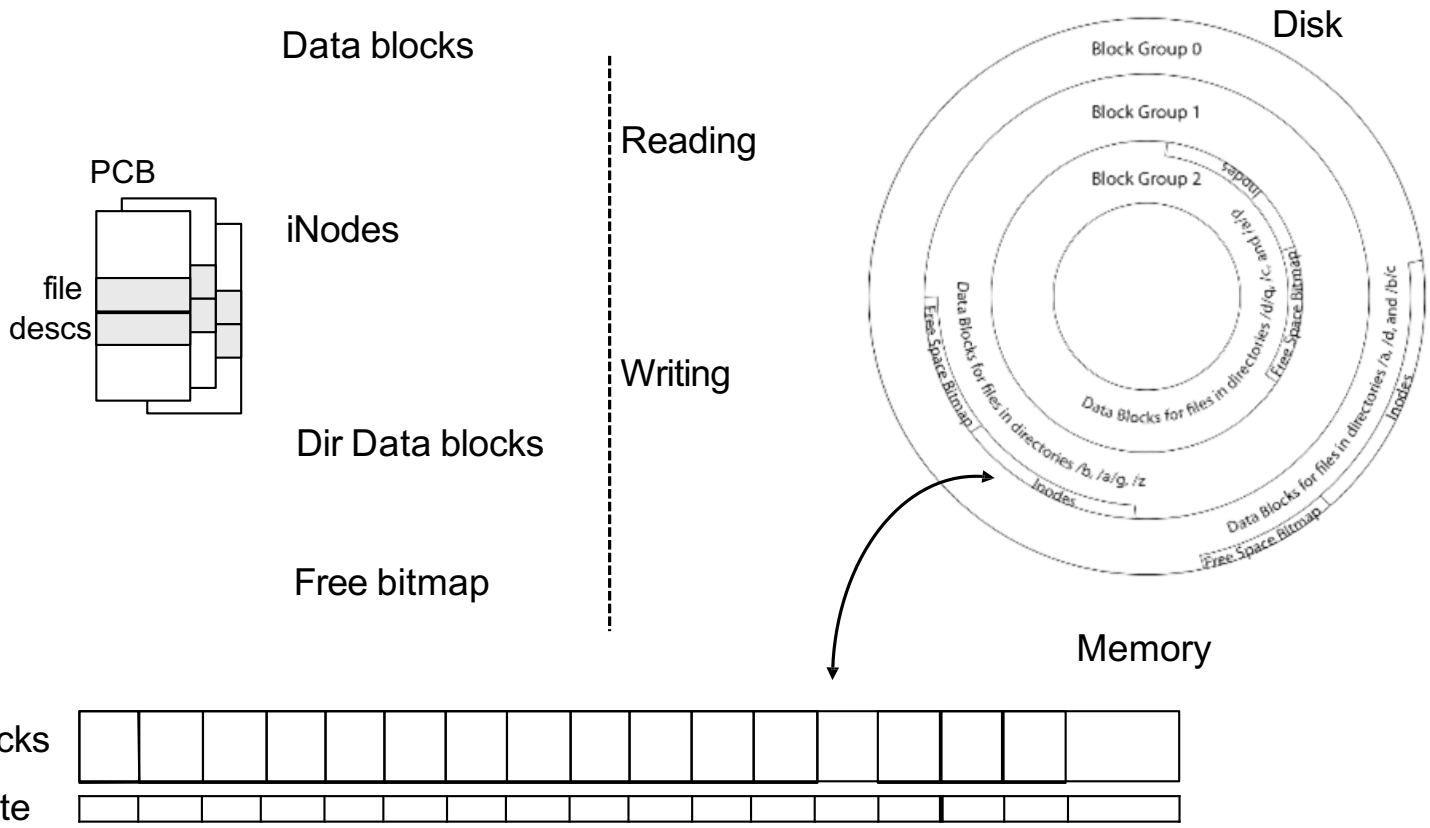
- Recall: kernel *must* copy disk blocks to main memory to access their contents and to modify parts of them for file writes

- Idea: exploit temporal locality by caching disk data in memory
  - Disk blocks: Mapping from block address → disk content
  - Name translations: Mapping from paths → inodes

- Buffer Cache: Memory used to cache FS data, including disk blocks and metadata
  - Can contain “dirty” blocks (with modifications not on disk)

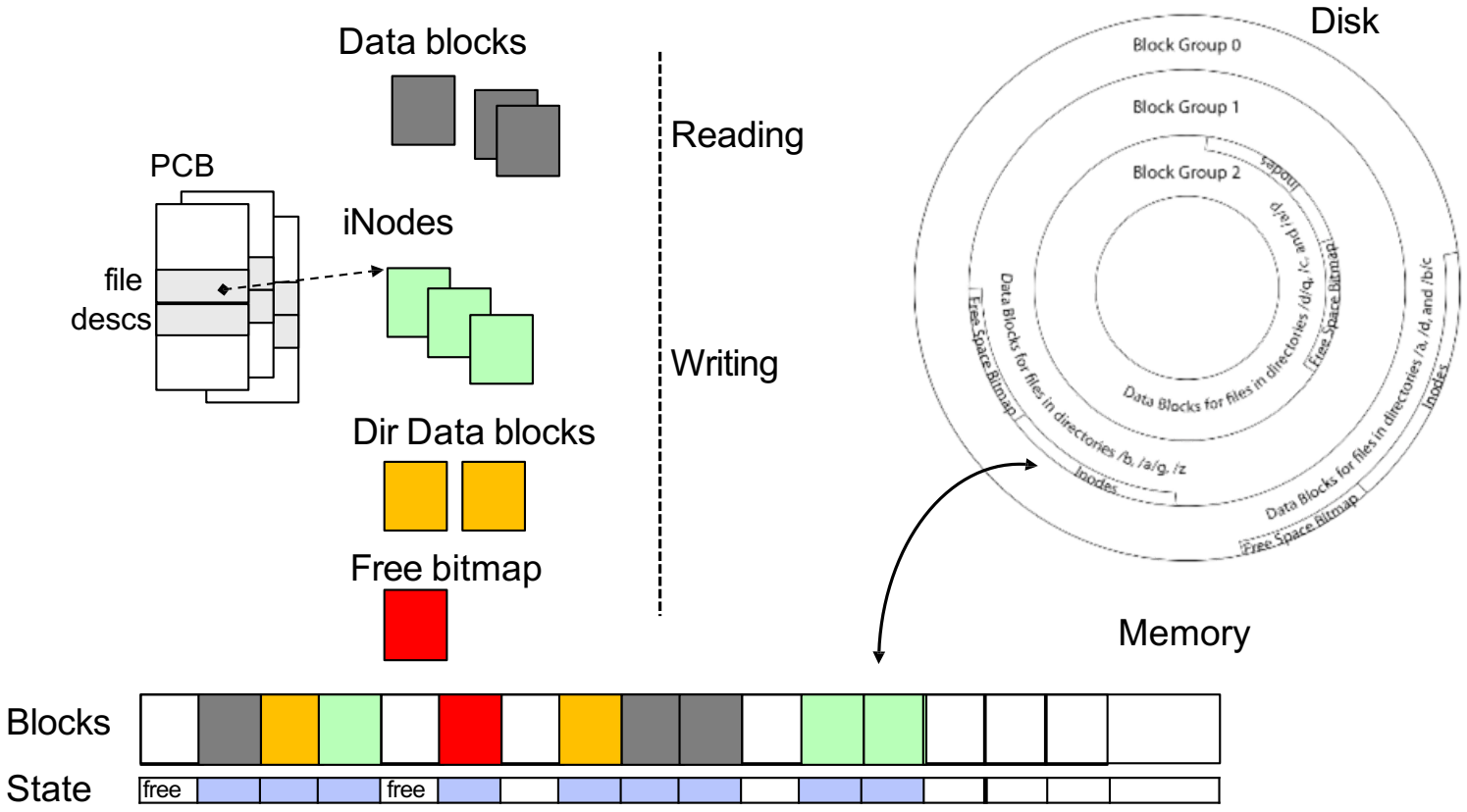
File system buffer cache

OS implements a cache of disk blocks for efficient access to data, directories, inodes, freemap



File system buffer cache

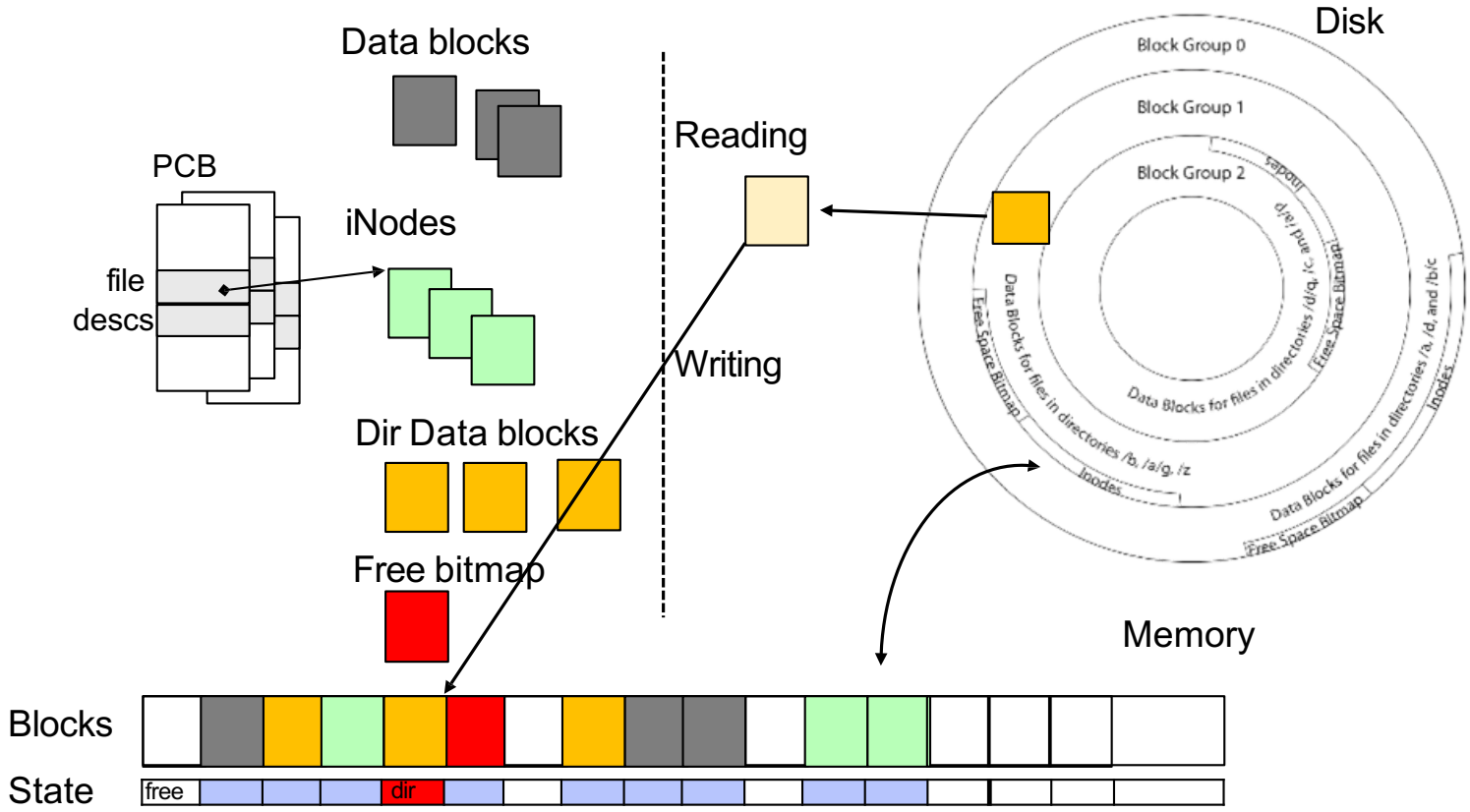
OS implements a cache of disk blocks for efficient access to data, directories, inodes, freemap



File system buffer cache: open()

Directory lookup (repeat as needed):

- load block of directory
- search for map

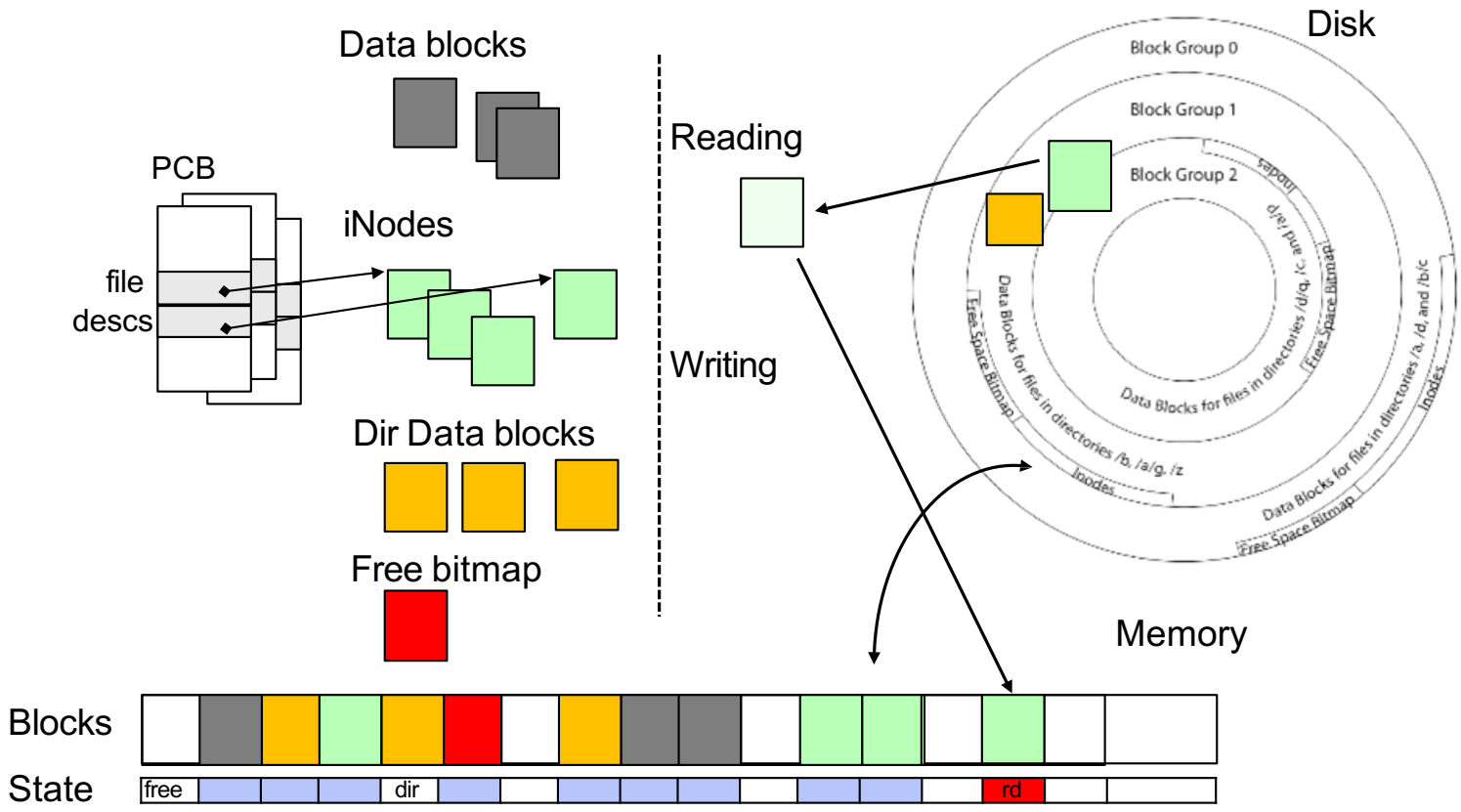


File system buffer cache: open()

Directory lookup (repeat as needed):

- load block of directory
- search for map

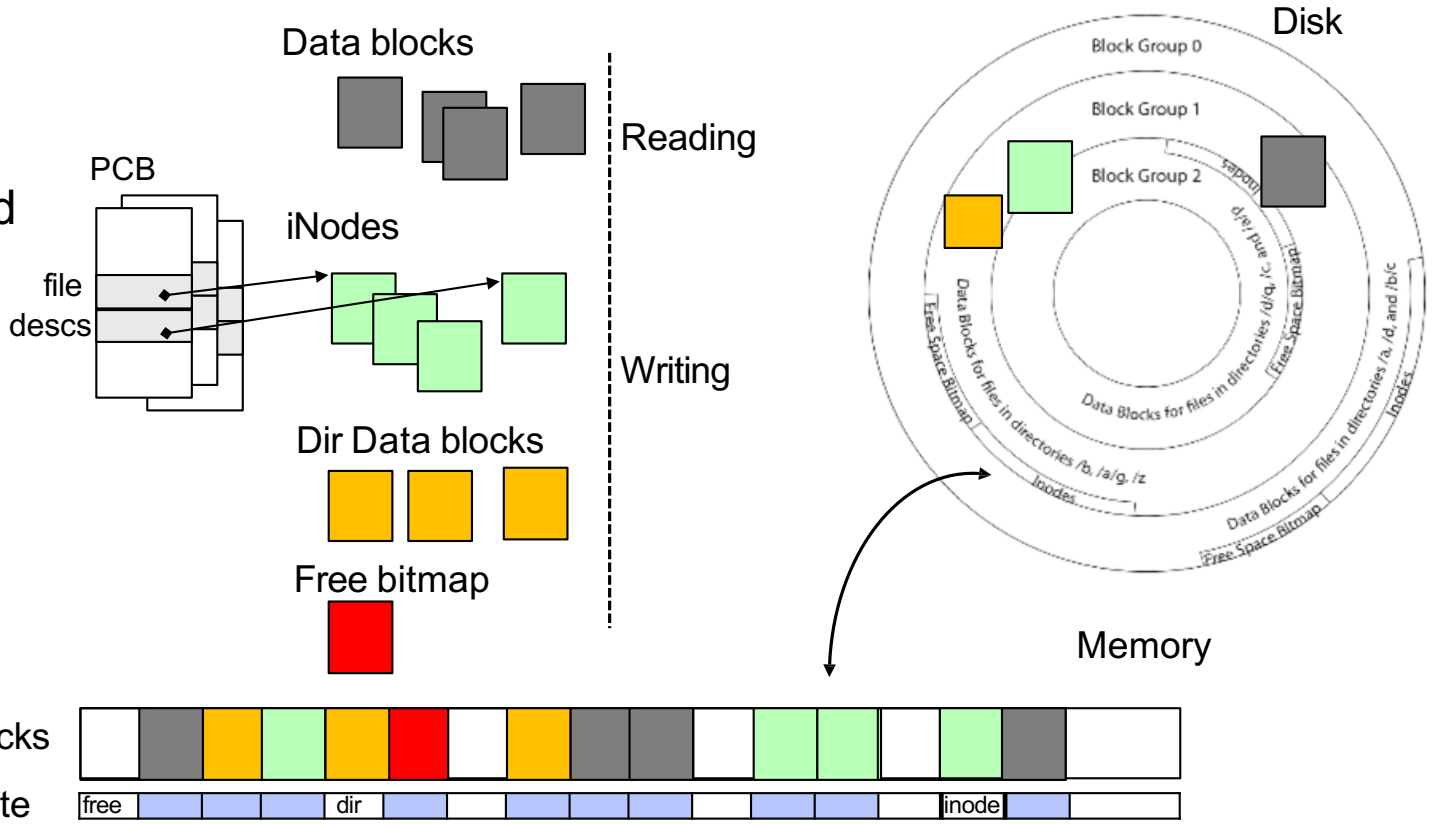
Create reference in open file descriptor



File system buffer cache: read()

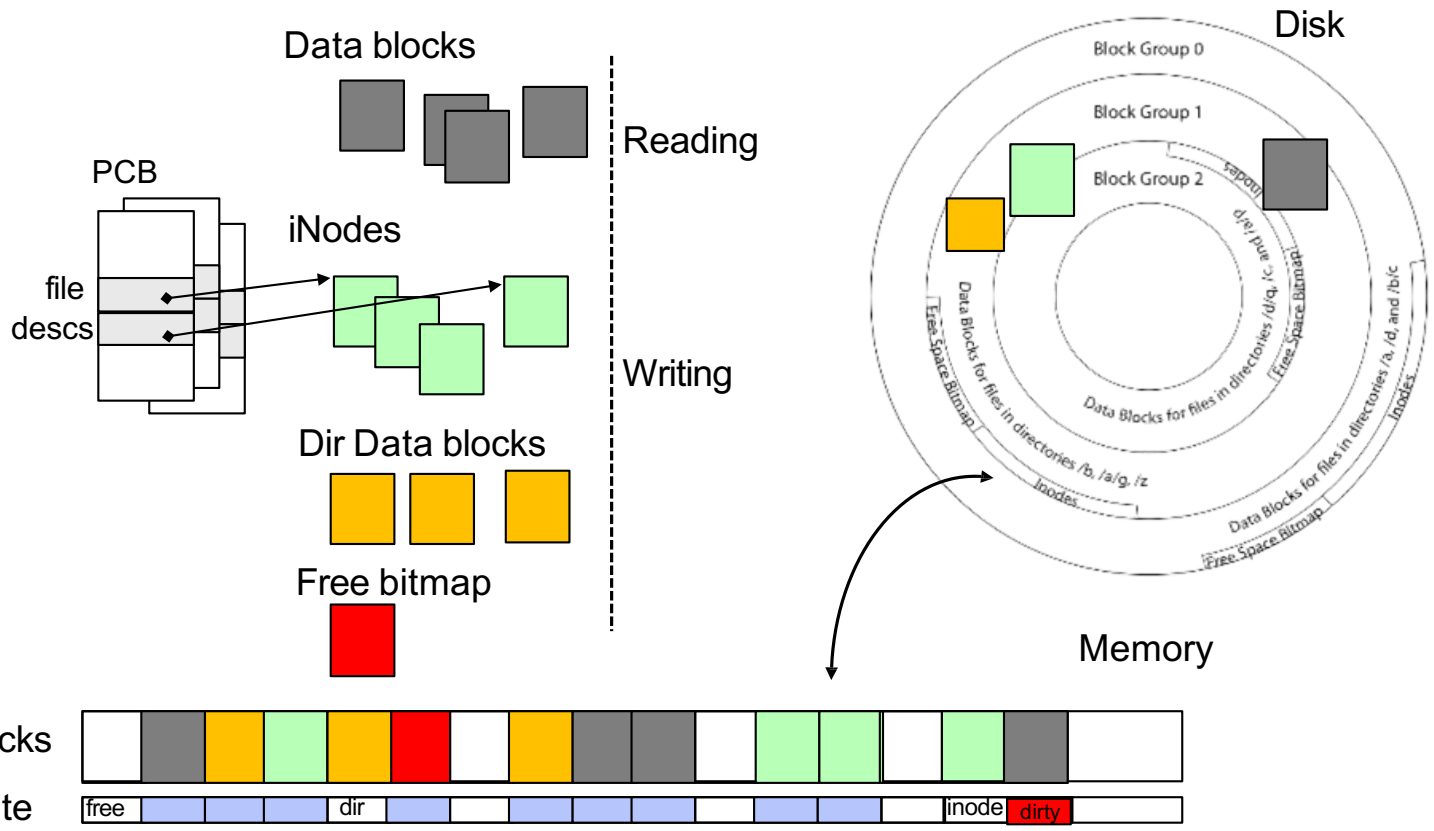
Read Process

- From inode, traverse index structure to find data block
- load data block
- copy all or part to read data buffer



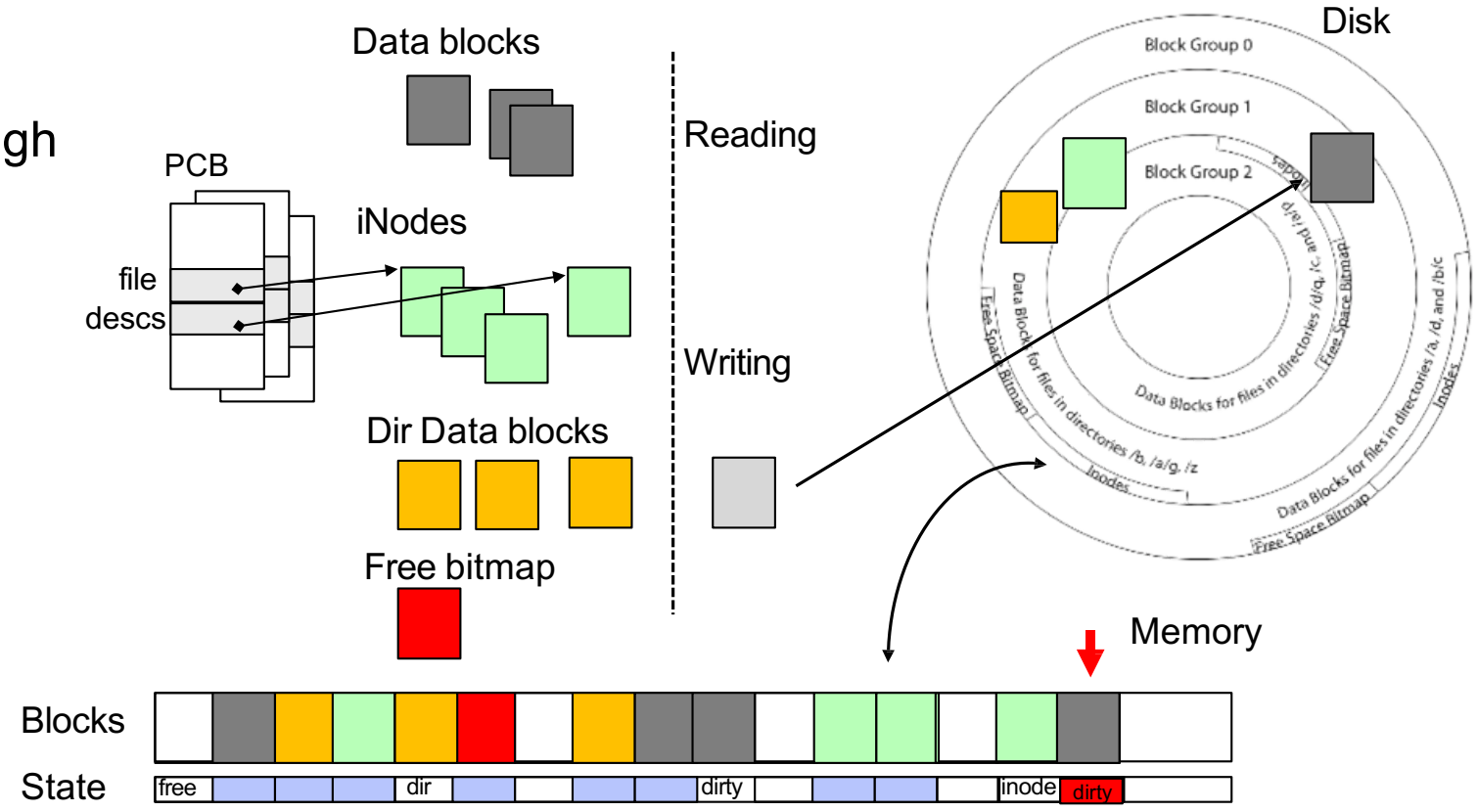
File system buffer cache: write()

Similar to read, but may allocate new blocks (update free map); blocks later need to be written back to disk; what about inode?



File system buffer cache: eviction?

Blocks being written back to disk go through a transient state



Buffer cache: replacement policy

- Preferred policy? LRU
  - Can afford overhead full LRU implementation
  - Advantages:
    - Works very well for name translation
    - Works well in general as long as memory is big enough to accommodate a host's working set of files.
  - Disadvantages:
    - Fails when some application scans through file system, thereby flushing the cache with data used only once
    - Example: `find . –exec grep foo {} \;`
- Other Replacement Policies?
  - Some systems allow applications to request other policies
  - Example, 'Use Once':
    - File system can discard blocks as soon as they are used

Buffer cache: size

- How much memory should the OS allocate to the buffer cache vs virtual memory?
  - Too much memory to the file system cache ⇒ won't be able to run many applications
  - Too little memory to file system cache ⇒ many applications may run slowly (disk caching not effective)
  - Solution: adjust boundary dynamically so that the disk access rates for paging and file access are balanced

File system prefetching

- Read Ahead Prefetching: fetch sequential blocks early
  - Key Idea: exploit fact that most common file access is sequential by prefetching subsequent disk blocks ahead of current read request
  - Elevator algorithm can efficiently interleave prefetches from concurrent applications

- How much to prefetch?
  - Too much prefetching imposes delays on requests by other applications
  - Too little prefetching causes many seeks

Delayed writes

- Buffer cache is a writeback cache
- `write()` copies data from user space to kernel buffer cache
  - Quick return to user space

- `read()` is fulfilled by the cache, so reads see the results of writes
  - Even if the data has not reached disk

- When does data from a write syscall finally reach disk?
  - When the buffer cache is full (e.g., we need to evict something)
  - When the buffer cache is flushed periodically (in case we crash)

Advantage of delayed writes

- Latency: return to user quickly without writing to disk!
- Disk scheduler can efficiently order lots of requests
  - Elevator Algorithm can rearrange writes to avoid random seeks
- Delay block allocation:
  - May be able to allocate multiple blocks at same time for file, keep them contiguous
- Some files never actually make it all the way to disk
  - Many short-lived files!

Buffer caching vs. Demand paging

- Replacement policy?
  - Demand paging: LRU is infeasible; use approximation (like Clock)
  - Buffer cache: LRU is OK

- Eviction policy?
  - Demand paging: evict not-recently-used pages when memory is close to full
  - Buffer cache: write back dirty blocks periodically, even if used recently
    - Why? To minimize data loss in case of a crash

Dealing with persistent state

- Buffer cache writes back dirty blocks periodically, even if used recently
  - Why? To minimize data loss in case of a crash
  - Linux does periodic flush every 30 seconds
  - Applications can use `fsync` to force flushing a file

- Not foolproof! Can still crash with dirty blocks in the cache
  - What if the dirty block was for a directory?
    - Lose pointer to file's inode (leak space)
    - File system now in an inconsistent state

Boom!


HANDLE*** Address 8016a950 has base at 80100000
.6.2 irql:1f SYSVER 0xf0000565
Name
Dll Base DateStmp - Name
ntoskrnl.exe
atapi.sys
Disk.sys
Ntfs.sys
NTice.sys
Cdrom.SYS
KSecDD.SYS
win32k.sys
Cdfs.SYS
nbf.sys
netbt.sys
afd.sys
Parport.SYS

File system reliability

Important “ilities”

- Availability
  - The probability that the system can accept and process requests
- Durability
  - The ability of a system to recover data despite faults
- Reliability
  - The ability of a system or component to perform its required functions under stated conditions for a specified period of time (IEEE definition)

File system reliability

- What can happen if disk loses power or software crashes?
  - Some operations in progress may complete
  - Some operations in progress may be lost
  - Overwrite of a block may only partially complete

- File system needs durability (as a minimum!)
  - Data previously stored can be retrieved (maybe after some recovery step), regardless of failure

- But durability is not quite enough…!

Storage reliability problem

- Single logical file operation can involve updates to multiple physical disk blocks
  - inode, indirect block, data block, bitmap, …
  - With sector remapping, single update to physical disk block can require multiple (even lower level) updates to sectors

- At a physical level, operations complete one at a time
  - Want concurrent operations for performance

- How do we guarantee consistency regardless of when crash occurs?

Threats to reliability

- Interrupted operation
  - Crash or power failure in the middle of a series of related updates may leave stored data in an inconsistent state
  - Example: transfer funds from one bank account to another
  - What if transfer is interrupted after withdrawal and before deposit?

- Loss of stored data
  - Failure of non-volatile storage media may cause previously stored data to disappear or be corrupted

Two reliability approaches

Careful ordering and recovery

FAT & FFS + (fsck)

Each step builds structure,
Data block ⇐ inode ⇐ free ⇐ directory

Last step links it in to rest of FS

"Recover" operation scans structure looking for incomplete actions

Versioning and copy-on-write

ZFS, …

Version files at some granularity

Create new structure linking back to unchanged parts of old one

Last step is to declare that the new version is ready

Reliability approach #1: careful ordering

- Sequence operations in a specific order
  - Careful design to allow sequence to be interrupted safely

- Post-crash recover operation
  - Read data structures to see if there were any operations in progress
  - Clean up/finish as needed

- Approach taken by
  - FAT and FFS (fsck) to protect filesystem structure/metadata
  - Many app-level recovery schemes (e.g., Word, emacs autosaves)

Berkeley FFS: create a file

Normal operation:

- Allocate data block
- Write data block
- Allocate inode
- Write inode block
- Update bitmap of free blocks and inodes
- Update directory with file name → inode number
- Update modify time for directory

Recovery:

- Scan inode table
- If any unlinked files (not in any directory), delete or put in lost & found dir
- Compare free block bitmap against inode trees
- Scan directories for missing update/access times

Time proportional to disk size

Reliability approach #2: copy-on-write file layout

- Create a new version of the file with the updated data
  - Reuse blocks that don't change much of what is already in place
  - Only point to the new version when fully done writing it

- Seems expensive!
  - But updates can be batched, and many disk writes can occur in parallel

- Approach taken in network file server appliances
  - NetApp's Write Anywhere File Layout (WAFL)
  - ZFS (Sun/Oracle) and OpenZFS

More general reliability solutions

- Use *transactions* for atomic updates
  - Ensure that multiple related updates are performed atomically
  - I.e., if a crash occurs in the middle, the state of the systems reflects either all or none of the updates
  - Most modern file systems use transactions internally to update metadata
  - Many applications also implement their own transactions
- Use *redundancy* for media failures
  - Redundant representation on one storage device (Error Correcting Codes)
  - Replication across devices (e.g., RAID disk array)

Transactions

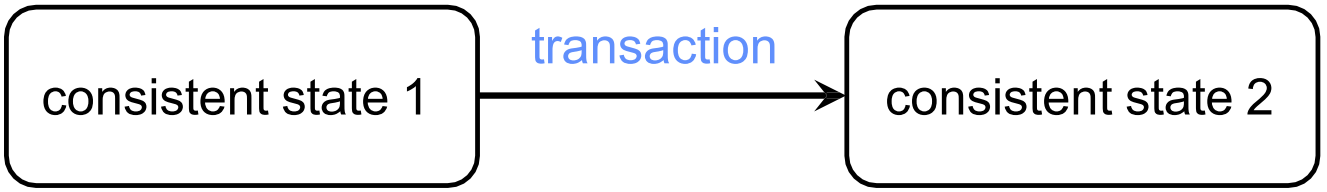
- Closely related to critical sections for manipulating shared data structures

- They extend concept of atomic update from memory to stable storage
  - Atomically update multiple persistent data structures

- Many ad-hoc approaches
  - FFS carefully ordered the sequence of updates so that if a crash occurred while manipulating directory or inodes, the disk scan on reboot would detect and recover the error (fsck)
  - Applications use temporary files and rename

Key concept: transaction

- A *transaction* is an atomic sequence of reads and writes that takes the system from one consistent state to another.



- Recall: Code in a critical section appears atomic to other threads
- Transactions extend the concept of atomic updates from *memory* to *persistent storage*

Typical structure

- Begin a transaction – get a transaction ID

- Do a bunch of updates
  - If any fail along the way, roll-back
  - Or, if any conflicts with other transactions, roll-back

- Commit the transaction

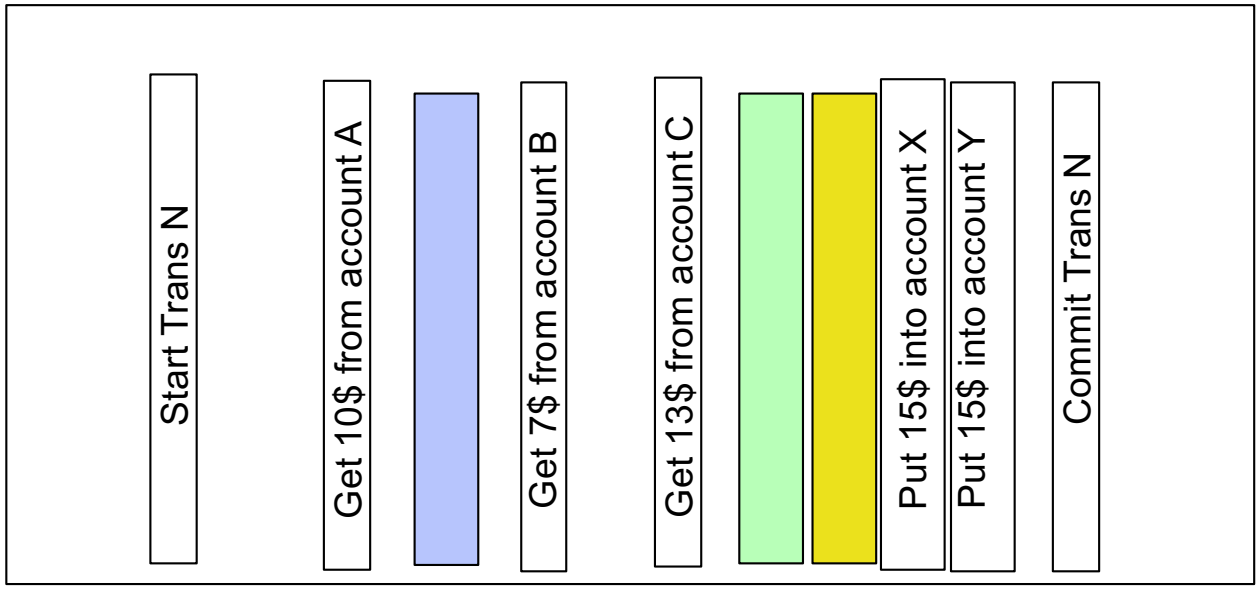
“Classic” example: transactions in SQL databases

Transfer $100 from Alice’s account to Bob’s

```
BEGIN;      --BEGIN TRANSACTION

UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';

UPDATE branches SET balance = balance - 100.00 WHERE
  name = (SELECT branch_name FROM accounts WHERE name = 'Alice');

UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';

UPDATE branches SET balance = balance + 100.00 WHERE
  name = (SELECT branch_name FROM accounts WHERE name = 'Bob');

COMMIT;     --COMMIT WORK
```

Useful tool: a log

- One simple action is atomic – write/append a basic item
- Use that to seal the commitment to a whole series of actions



Transactional file systems

- Better reliability through use of log
  - Changes to all FS data structures are treated as transactions
  - A transaction is committed once it is fully written to the log
    - Data forced to disk for reliability
    - Process can be accelerated with NVRAM
  - Although the actual file system data structures may not be updated immediately, data preserved in the log and replayed to recover
- Difference between “Log Structured” and “Journaled” file systems
  - In a Log Structured filesystem, all data stays in log form
  - In a Journaled filesystem, log only used for recovery

Journaling file systems

- Don’t modify data structures on disk directly
- Write each update as transaction recorded in a log
  - Commonly called a journal or intention list
  - Also maintained on disk (allocate specific blocks for it)
- Once changes are in the log, they can be safely applied to other data structures
  - E.g. modify inode pointers and directory entries
- Linux took original FFS-like file system (ext2) and added a journal to get ext3!
  - Some options: whether or not to write all data to journal or just metadata

Creating a file (no journaling yet)

Find free data block(s)

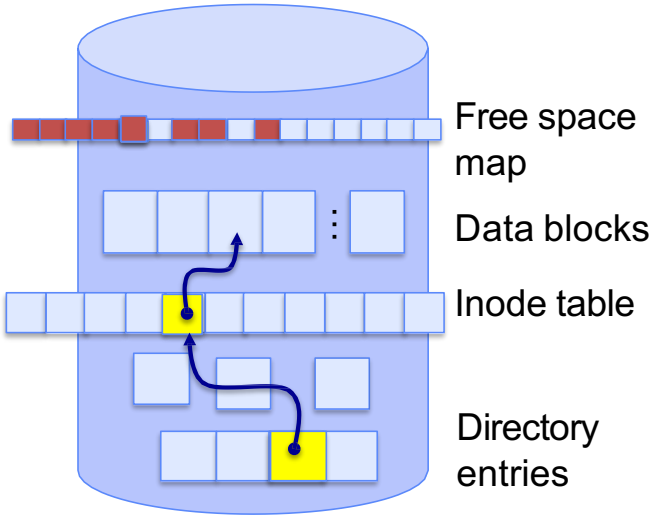
Find free inode entry

Find dirent insertion point

Write bitmap (i.e., mark used)

Write inode entry to point to block(s)

Write dirent to point to inode



Creating a file (with journaling)

Find free data block(s)

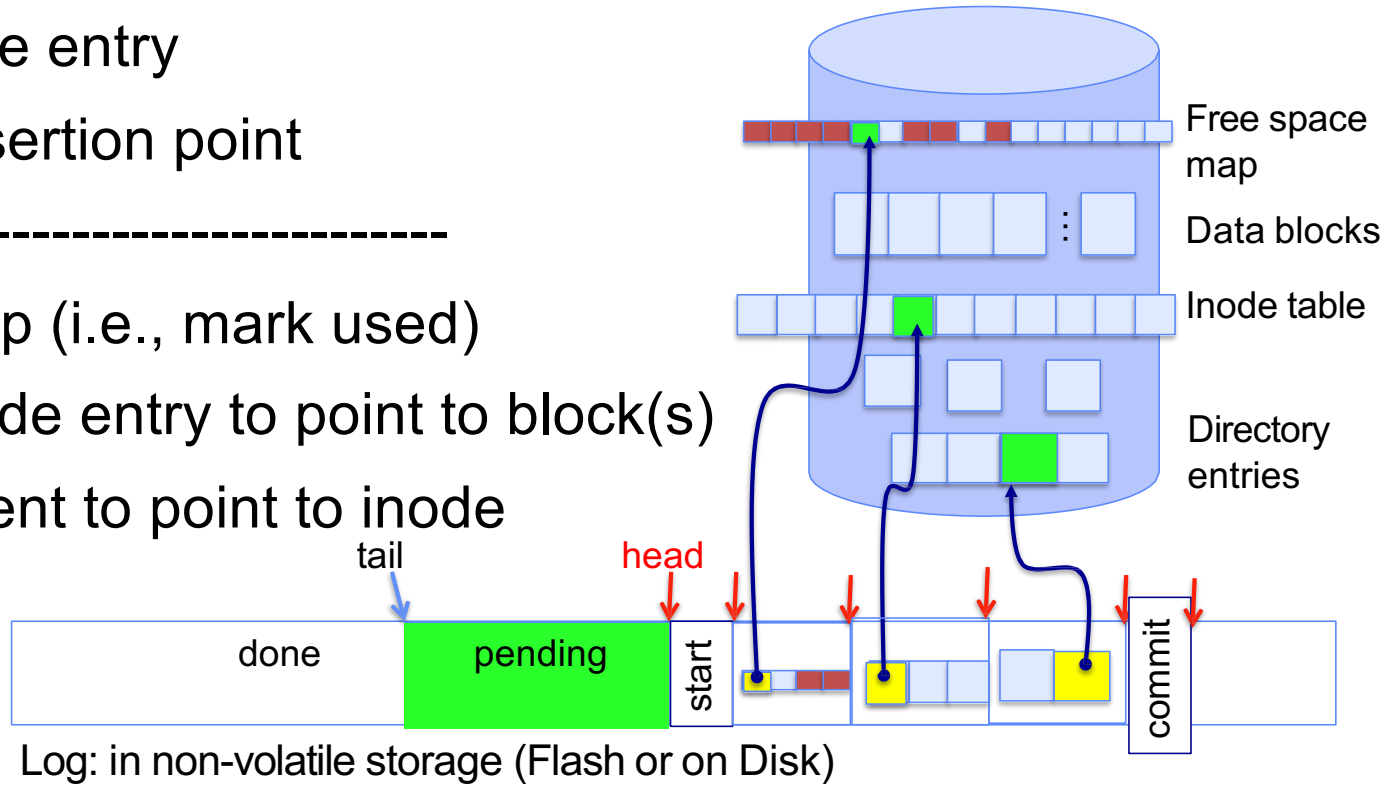
Find free inode entry

Find dirent insertion point

[log] Write map (i.e., mark used)

[log] Write inode entry to point to block(s)

[log] Write dirent to point to inode

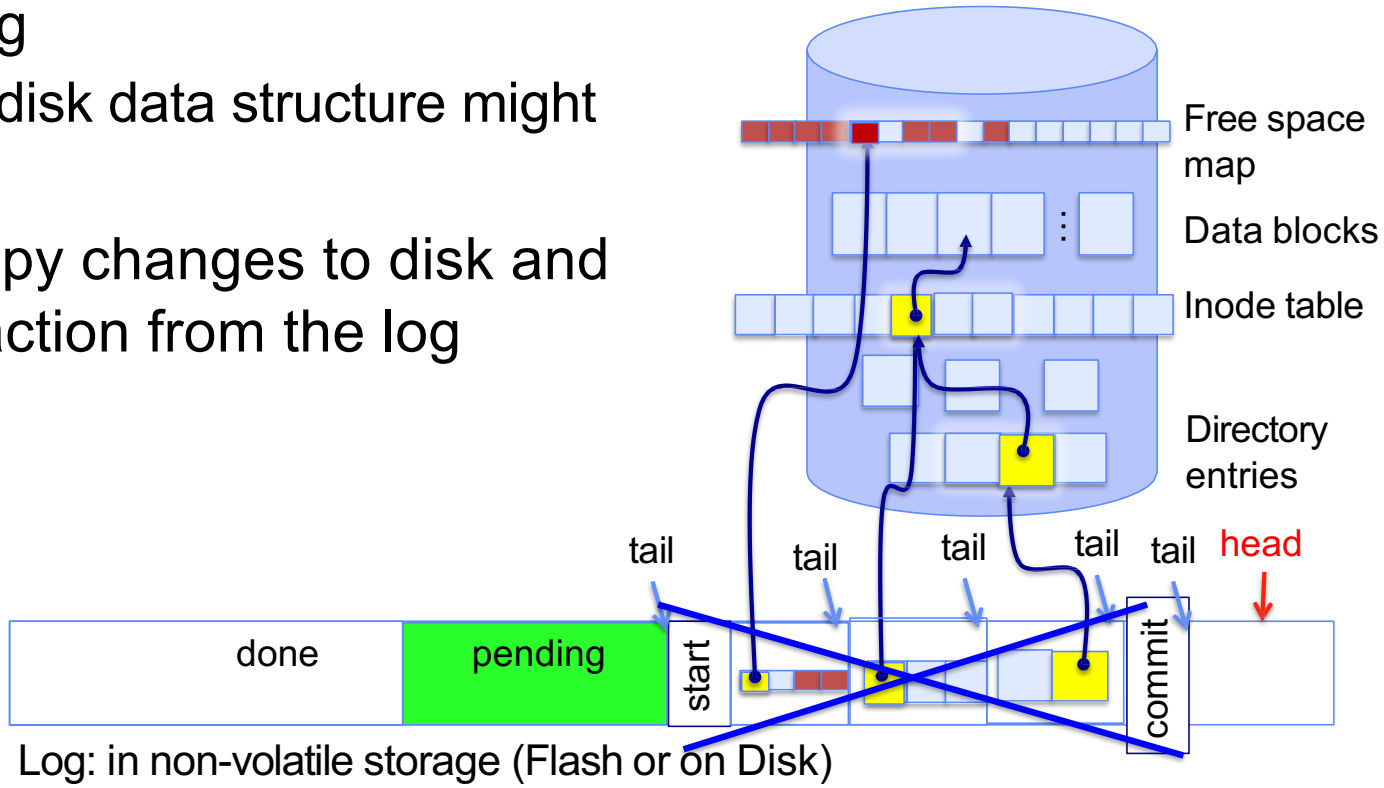


After commit, eventually replay transaction

All accesses to the file system first looks in the log

- Actual on-disk data structure might be stale

Eventually, copy changes to disk and discard transaction from the log



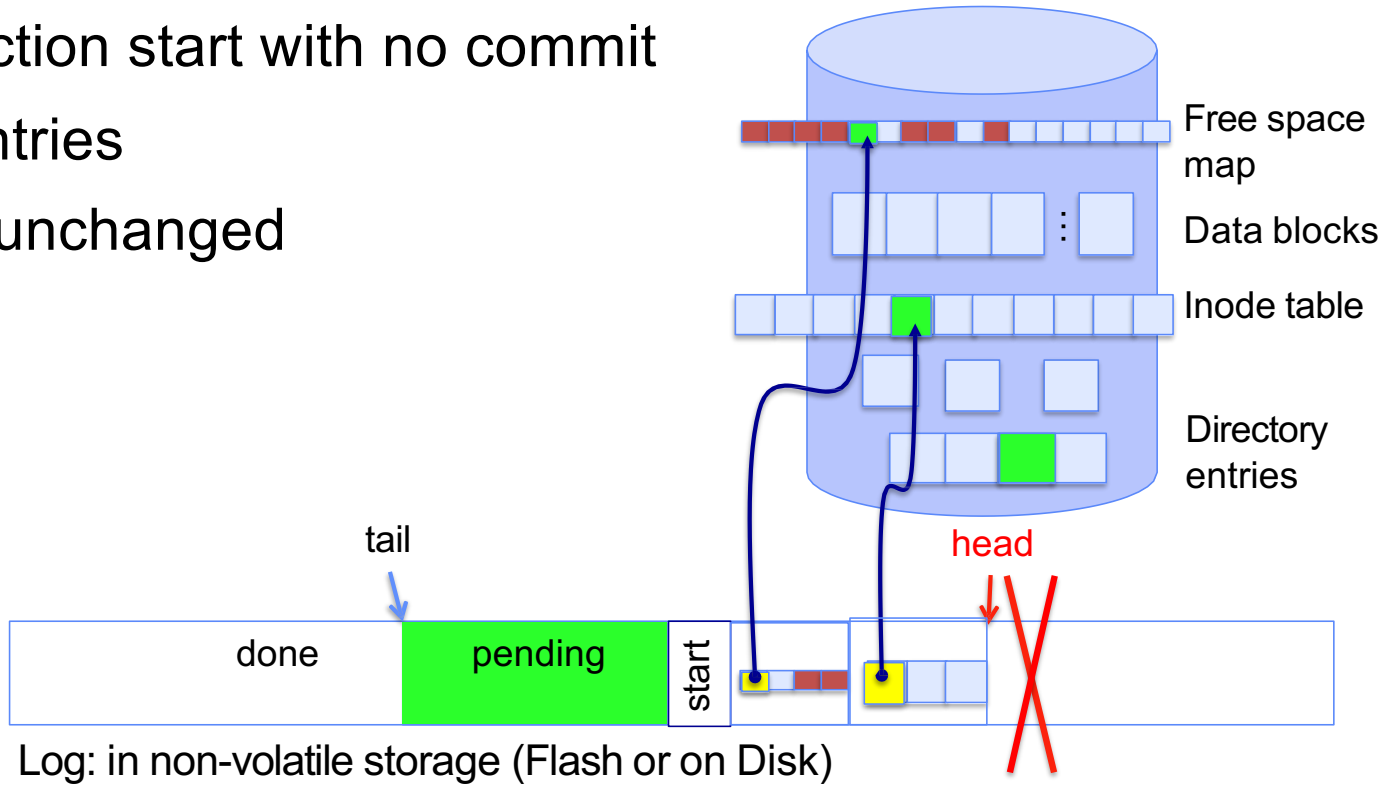
Crash recovery: discard partial transactions

Upon recovery, scan the log

Detect transaction start with no commit

Discard log entries

Disk remains unchanged



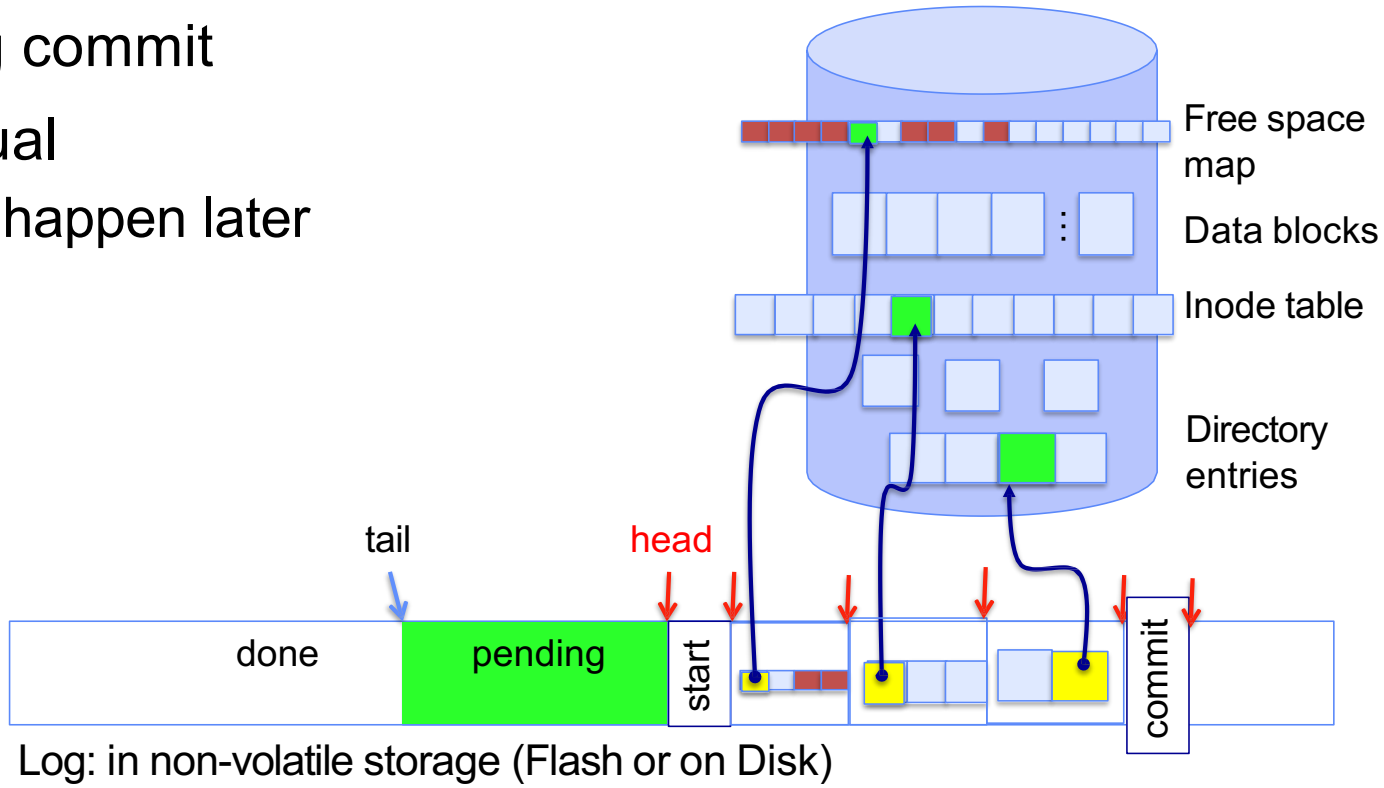
Crash recovery: keep complete transactions

Scan log, find start

Find matching commit

Redo it as usual

Or just let it happen later



Journaling summary

Why go through all this trouble?

- Updates atomic, even if we crash:
  - Update either gets fully applied or discarded
  - All physical operations *treated* as a *logical unit*

Isn’t this expensive?

- Yes! We're now writing all data twice (once to log, once to actual data blocks in target file)
- Modern filesystems journal metadata updates only
  - Record modifications to file system data structures
  - But apply updates to a file’s contents directly

Topic breakdown

- Virtualizing the CPU
  - Process Abstraction and API
  - Threads and Concurrency
  - Scheduling
- Virtualizing Memory
  - Virtual Memory
  - Paging
- Persistence
  - IO devices
  - File Systems
- Distributed Systems
  - Challenges with distribution
  - Data Processing & Storage

Topic breakdown

- Virtualizing the CPU
  - Process Abstraction and API
  - Threads and Concurrency
  - Scheduling
- Virtualizing Memory
  - Virtual Memory
  - Paging
- Persistence
  - IO devices
  - File Systems
- Distributed Systems
  - Challenges with distribution
  - Data Processing & Storage

Thanks